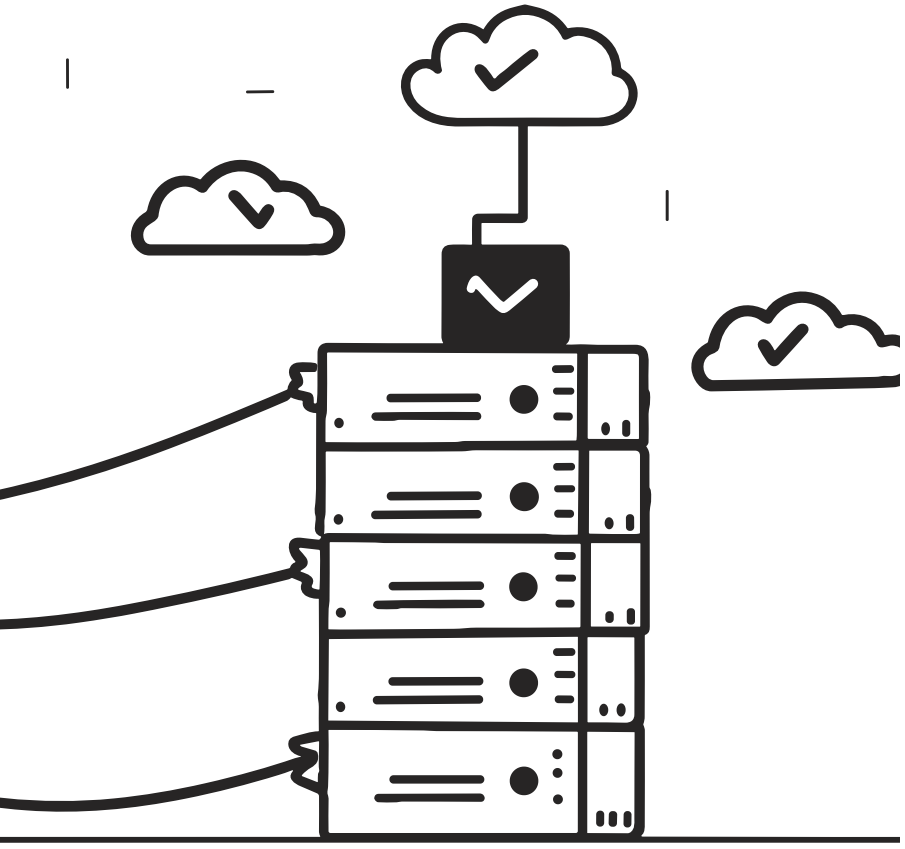


# Mini-Projet : Construction d'un Pipeline ETL de A à Z

MODULE CPPE835 | CONCEPTION DES PIPELINES | SEMESTRE 8

Ce mini-projet est conçu pour tester l'ensemble des compétences acquises dans le module Conception des Pipelines. Il simule une situation professionnelle réelle et doit être réalisé en autonomie en quelques heures.

### EID3.5

Extraction de données multi-sources

### EID3.6

Transformation et enrichissement

### EID3.9

Chargement et historisation

# Contexte & Mise en Situation Professionnelle

## Votre employeur

**DataRetail SA** — société de e-commerce de taille moyenne

Vous venez d'être recruté(e) en tant que **Data Engineer**.

> « Nous avons des données de ventes dispersées entre notre API interne, notre base de données PostgreSQL et un fichier CSV exporté chaque nuit. Nous avons besoin d'un pipeline ETL qui centralise tout ça dans notre entrepôt de données. »
> — Le Directeur Technique

## Votre mission

Concevoir et développer un pipeline ETL complet, de l'extraction des données jusqu'au chargement dans la base de destination, avec historisation des traitements.

→ **Extraire**

Depuis 3 sources hétérogènes

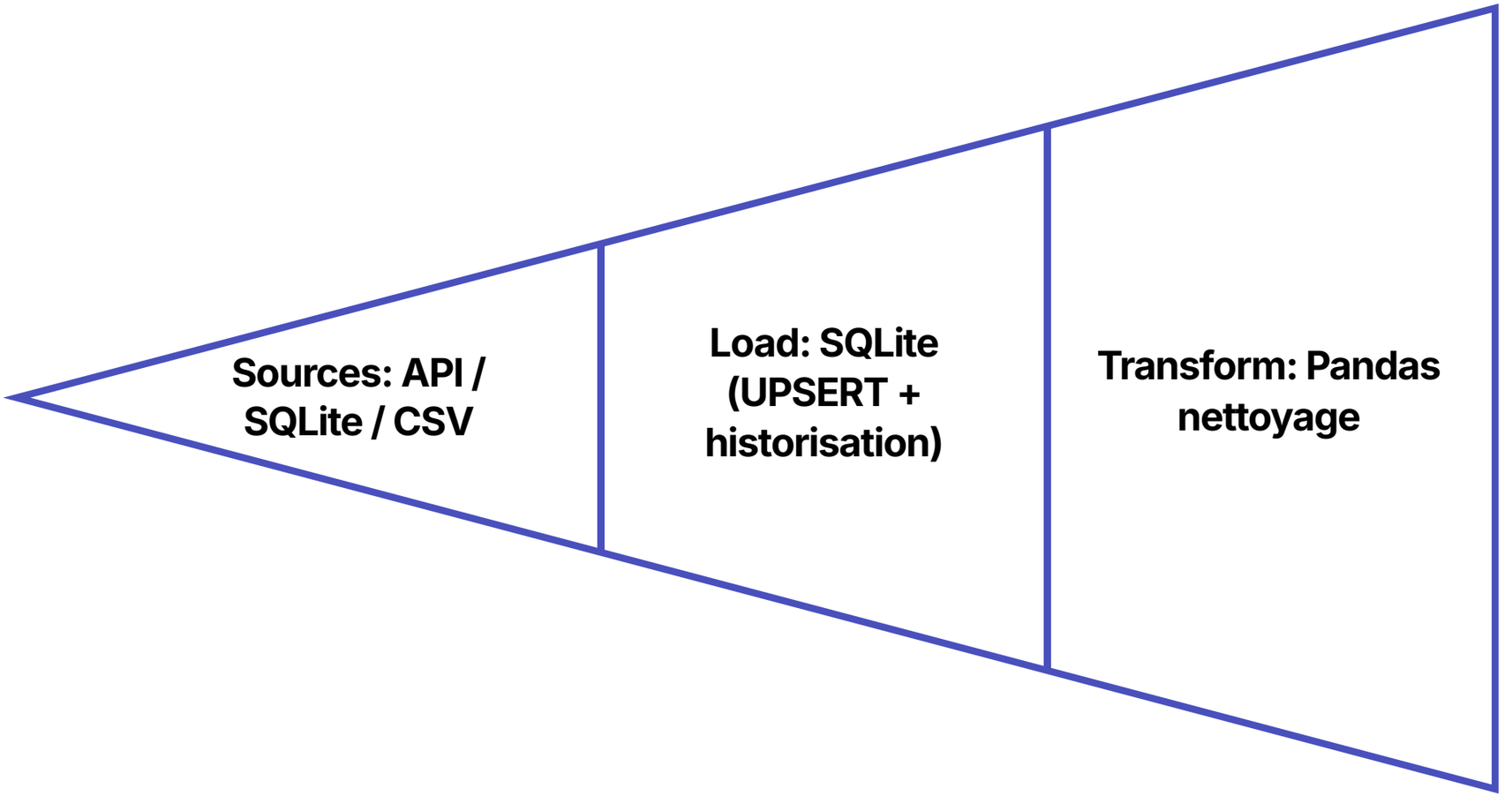
→ **Transformer**

Nettoyer, enrichir, unifier

→ **Charger**

Vers l'entrepôt de données

# Architecture Cible du Pipeline



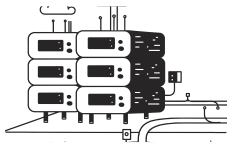

### Sources d'entrée

- API REST publique (JSONPlaceholder)
- Base de données SQLite locale
- Fichier CSV exporté nuitamment

### Livrable attendu

- Script Python fonctionnel `etl_pipeline.py`
- Rapport de conception (1 page A4)
- Fichier log complet du pipeline

# Données Fournies



### Fichier CSV — `orders.csv`

500 commandes clients avec les colonnes :

- `order_id`
- `customer_id`
- `product_name`
- `amount`
- `order_date`

### Base SQLite — `customers.db`

Table `customers` avec les colonnes :

- `customer_id`
- `email`
- `signup_date`
- `country`

### API Publique — JSONPlaceholder

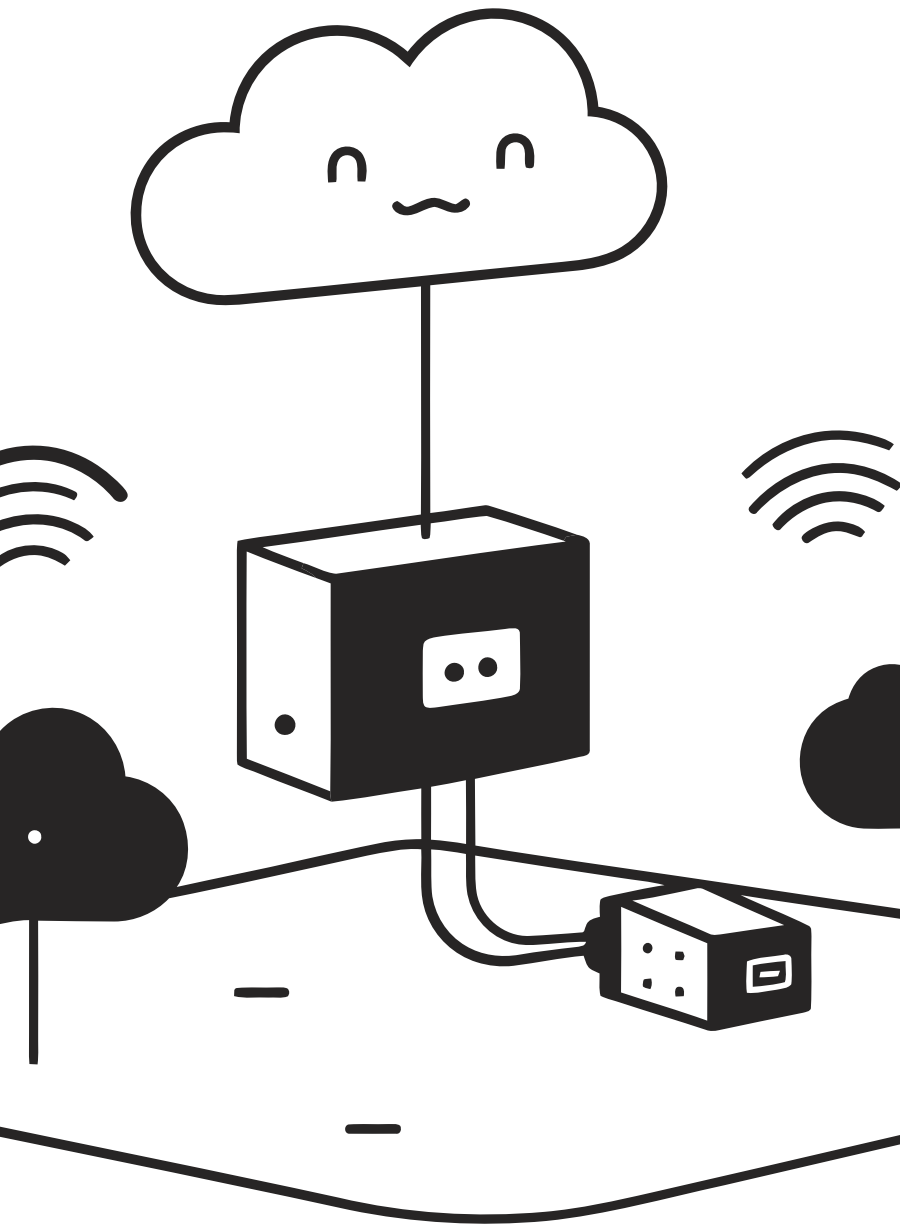
Endpoint : `https://jsonplaceholder.typicode.com/users`

Retourne 10 utilisateurs JSON simulant des comptes premium.

### Base de destination — `datawarehouse.db`

Table cible à créer :

`fact_orders_enriched`



# Partie 1 : Extract (E)

**Objectif :** Extraire les données des 3 sources en mémoire sans erreur bloquante.

| | |
|---|---|
| 1 | **Tâche 1.1 — Requêtage de l'API**<br>• Requêter `https://jsonplaceholder.typicode.com/users`<br>• Gérer les erreurs HTTP (`try/except`, status code)<br>• Stocker le résultat dans un DataFrame Pandas |
| 2 | **Tâche 1.2 — Requêtage SQLite**<br>• Connexion à `customers.db` avec SQLAlchemy ou sqlite3<br>• Requête SQL optimisée (colonnes nécessaires uniquement)<br>• Charger dans un DataFrame |
| 3 | **Tâche 1.3 — Lecture du fichier CSV**<br>• Lire `orders.csv` avec Pandas<br>• Détecter et signaler les valeurs manquantes<br>• Afficher un résumé du dataset (shape, types, nulls) |

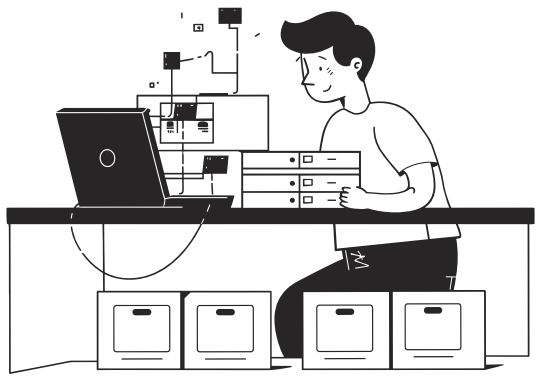
✅ **Critère de réussite :** Les 3 sources sont chargées en mémoire sans erreur bloquante.

# Partie 2 : Transform (T)

**Objectif :** Nettoyer, enrichir et unifier les données issues des 3 sources.

## 1 Nettoyage

- Traiter les valeurs manquantes dans `orders.csv` (stratégie à justifier)
- Normaliser les formats de date (`order_date`, `signup_date`) → `datetime`
- Vérifier l'unicité des `order_id`



## 2 Enrichissement

- Jointure entre `orders` et `customers` sur `customer_id`
- Créer `days_since_signup` (jours entre inscription et commande)
- Créer `is_premium` (True si `customer_id` dans les données API)
- Ajouter `etl_timestamp` avec l'heure courante du traitement

## 3 Validation

- Vérifier qu'aucune ligne ne dépasse 100% de valeurs nulles
- Afficher les 5 premières lignes du DataFrame final

✅ **Critère de réussite :** DataFrame `fact_orders_enriched` propre, typé, avec toutes les colonnes attendues.

# Partie 3 : Load (L)

**Objectif :** Charger les données transformées en base et historiser les traitements.

### Tâche 3.1 — Insertion SQLite

- Créer la table `fact_orders_enriched` si elle n'existe pas
- Stratégie **TRUNCATE-INSERT** (vider puis réinsérer)
- Bonus : implémenter un **UPSERT** sur `order_id`

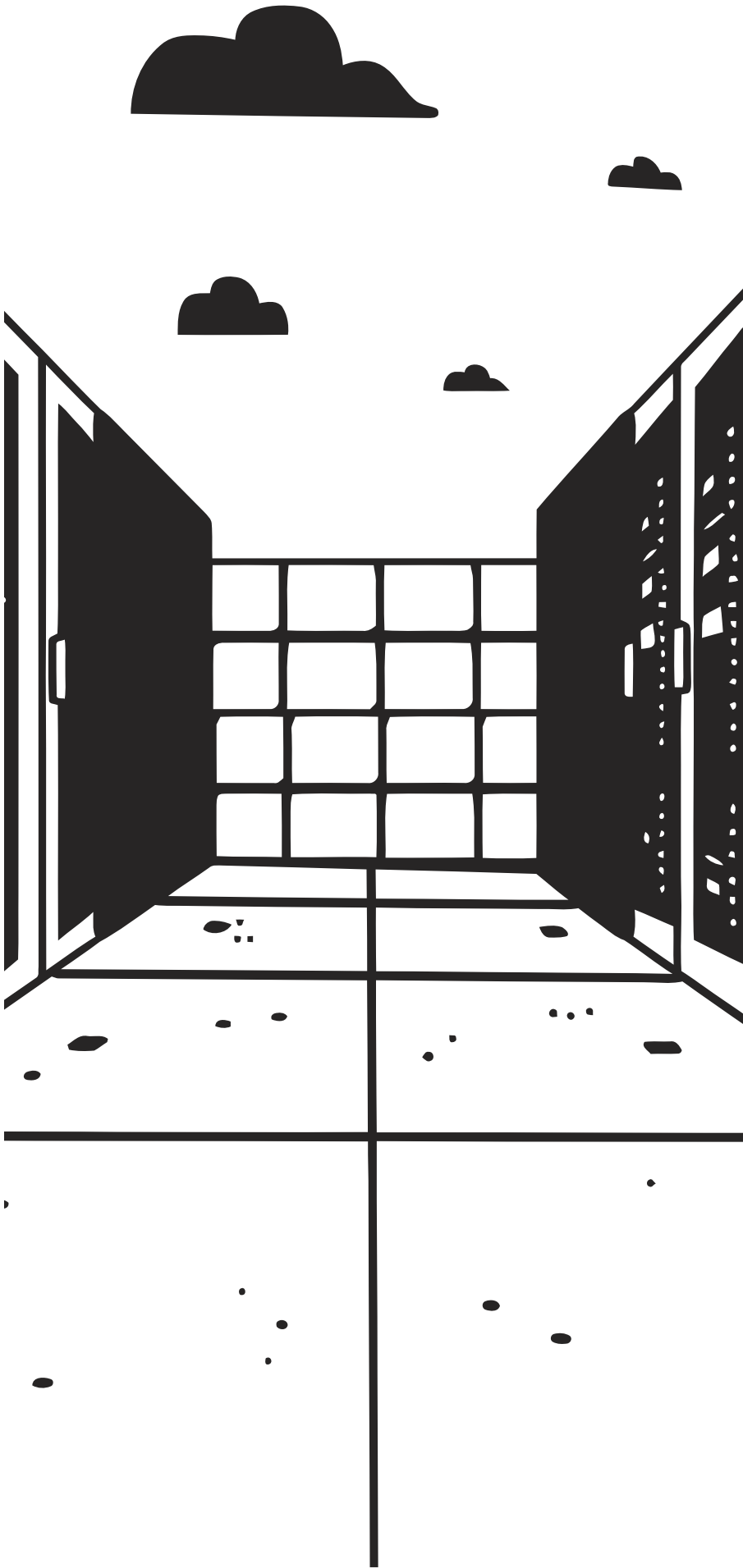
### Tâche 3.2 — Historisation

- Sauvegarder dans un CSV daté :
- `output/fact_orders_enriched_YYYYMMDD_HHMMSS.csv`
- Créer le dossier `output/` si inexistant

### Tâche 3.3 — Log du pipeline

- Date/heure d'exécution
- Nombre de lignes extraites par source
- Nombre de lignes chargées
- Éventuelles erreurs rencontrées

✅ **Critère de réussite :** La base de destination est alimentée, un fichier CSV daté est généré, le log est complet.



# Partie Bonus : Big Data (Optionnel)

+2 POINTS BONUS

POUR LES PLUS AVANCÉS

## Option A — PySpark

Refactorisez la partie Transform en utilisant **PySpark** au lieu de Pandas :

- Créer une SparkSession locale
- Recharger le CSV via `spark.read.csv()`
- Reproduire les transformations en DataFrame Spark (`select`, `join`, `withColumn`)
- Comparer les performances (temps d'exécution) avec la version Pandas

## Option B — Streaming Simulé

Simulez un pipeline de streaming avec Python :

- Lire le CSV ligne par ligne avec un délai (simulant un flux Kafka/Kinesis)
- Traiter chaque « événement » individuellement
- Insérer en base au fil de l'eau (ligne par ligne)

# Rapport de Conception (1 Page)

En complément du code, remettre un rapport de **maximum 1 page A4** structuré comme suit :

## 1. Choix d'architecture

- Justifier le choix **ETL vs ELT** pour ce cas d'usage
- Décrire le schéma du pipeline en quelques lignes

## 2. Décisions techniques

- Pourquoi cette stratégie de chargement (TRUNCATE-INSERT ou UPSERT) ?
- Comment avez-vous géré les erreurs ?
- Comment avez-vous historisé les données ?

## 3. Limites & Améliorations possibles

- Qu'est-ce qui manque pour un pipeline de production réel ?
- Quelles solutions cloud utiliseriez-vous ? (AWS S3, Google Cloud Storage, Airflow...)

# Livrables Attendus

| Livrable | Format | Obligatoire | Points |
|---|---|---|---|
| Script Python principal `etl_pipeline.py` | `.py` | ✅ Oui | Requis |
| Fichier CSV historisé | `.csv` daté | ✅ Oui | Requis |
| Fichier de log `pipeline.log` | `.txt` / `.log` | ✅ Oui | Requis |
| Rapport de conception | PDF ou Word, 1 page | ✅ Oui | Requis |
| Base de destination `datawarehouse.db` | `.db` | ✅ Oui | Requis |
| Version PySpark ou Streaming (Bonus) | `.py` | ⭐ Bonus | +2 pts |

📦 **Format de rendu :** Dossier zippé nommé `NOM_Prenom_ETL_CPPE835.zip`

# Critères d'Évaluation

GRILLE DE NOTATION SUR 20 POINTS

SEUIL DE VALIDATION : 12/20

**4**

### Extract

3 sources extraites sans erreur bloquante

**5**

### Transform

Jointures, colonnes calculées, nettoyage correct

**4**

### Load

Insertion en base + CSV daté + log complet

**3**

### Qualité code

Lisibilité, gestion d'erreurs, commentaires

**2**

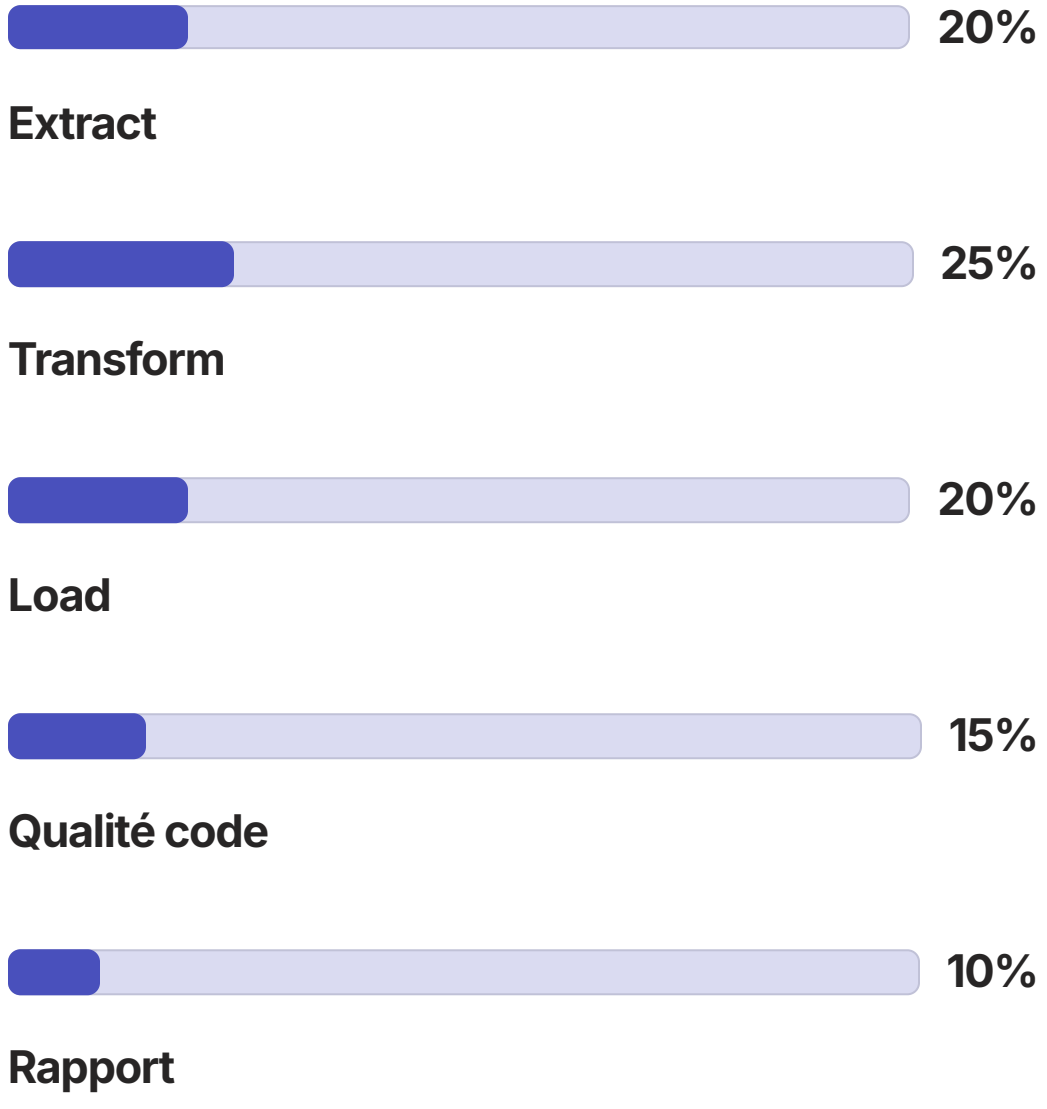
### Rapport

Justification des choix techniques

**+2**

### Bonus

PySpark ou Streaming simulé



# Ressources & Conseils

## Ressources utiles

### Documentation Pandas

pandas.pydata.org/docs/

### SQLAlchemy ORM

hackersandslackers.com

### ETL Best Practices

datachannel.co/blogs/etl-best-practices

### PySpark Streaming

george-jen.gitbook.io

### API JSONPlaceholder

jsonplaceholder.typicode.com

## Conseils pour réussir

### Testez source par source

Commencez par tester chaque source **indépendamment** avant d'assembler le pipeline complet.

### Utilisez des fonctions

Structurez votre code avec des **fonctions** pour chaque étape (extract, transform, load).

### Testez la gestion d'erreurs

Coupez temporairement la connexion API pour valider votre gestion des exceptions.

### Le log est votre allié

Le fichier log est votre meilleur outil pour déboguer un pipeline en production.

🚀 **Bonne chance !** Ce projet simule une vraie mission de Data Engineer — soyez rigoureux(se) et méthodique.